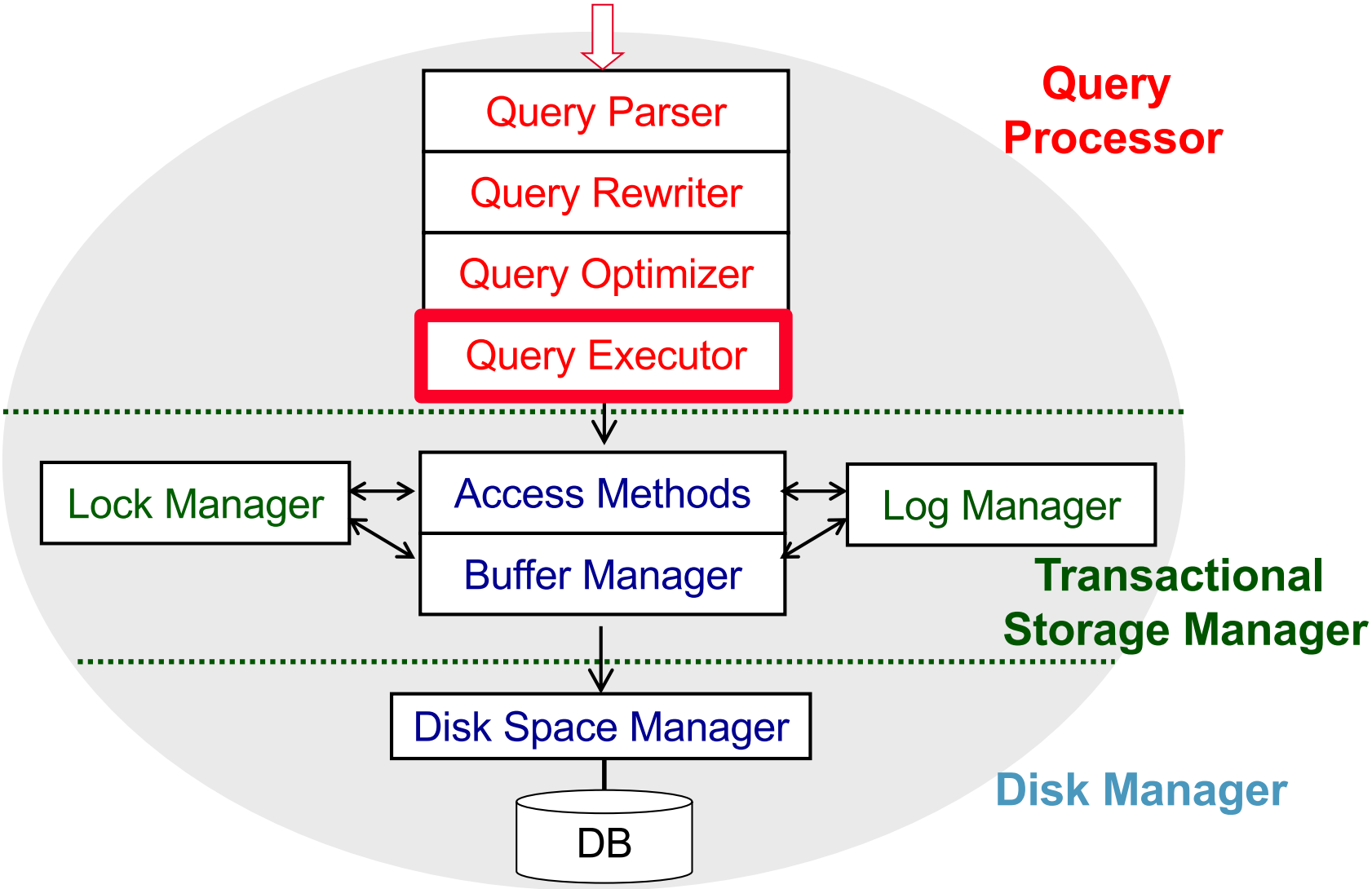
Evaluation of Relational Operations

Ioana Manolescu
Inria and Ecole Polytechnique



Slides Courtesy of R. Ramakrishnan, J. Gehrke and Y. Diao

Overview of Query Processing



Relational operators

- We will consider how to implement:
  - *Selection* (σ) (based on indexes seen in the last course)
  - *Sorting routine* (needed by many other operators)
  - *Join* ($\bowtie$) Allows to combine two relations.
  - *Projection* (π) Deletes unwanted columns from relation.

  - *Union* ($\cup$) Tuples in either reln. 1 or reln. 2.
  - *Intersection* ($\cap$) Tuples in both reln. 1 and reln. 2.
  - *Set-difference* ($-$) Tuples in reln. 1, but not in reln. 2.

  - *GROUP BY* and *Aggregation* (SUM, MIN, etc.)
- Multiple choices for each of the above

General goal: efficiently work on data that does not fit in memory

- Data larger than the available memory:
  - Clearly the situation when DBMS development started
  - Memory sizes have tremendously grown, but...
  - So have database sizes!
  - Also: concurrent executions for many clients

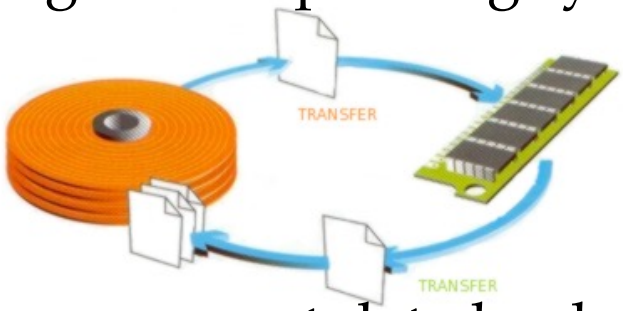
Recall:

- The DBMS server process **reserves part of the OS memory** and manages this memory by itself ("buffers")
- This is to avoid **arbitrary performance degradation through OS swap** (see next)

Working with data between stable storage and memory

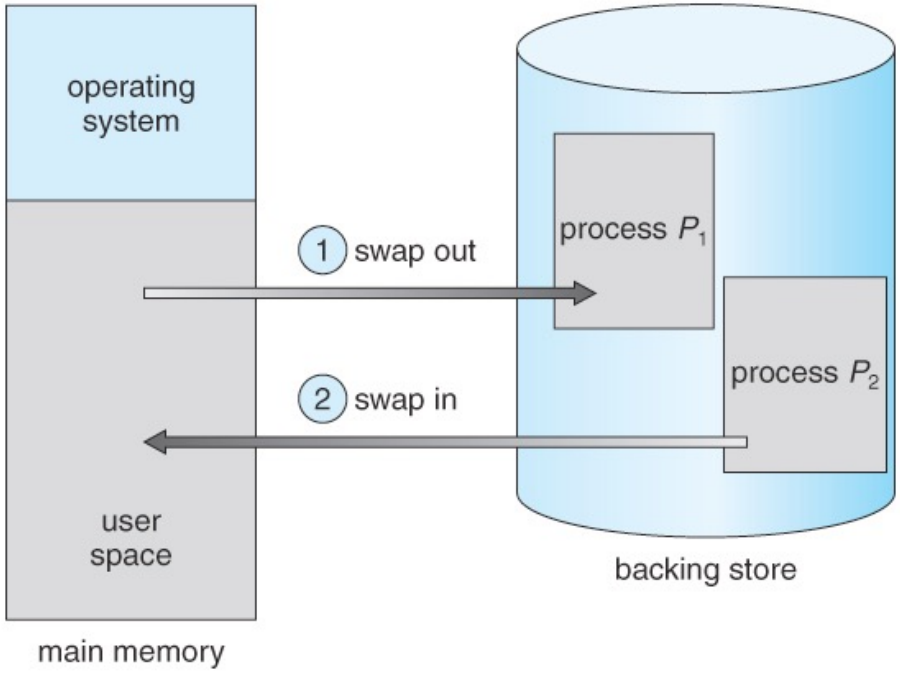
- Normal functioning of the operating system (OS):

Persistent storage, e.g., disk



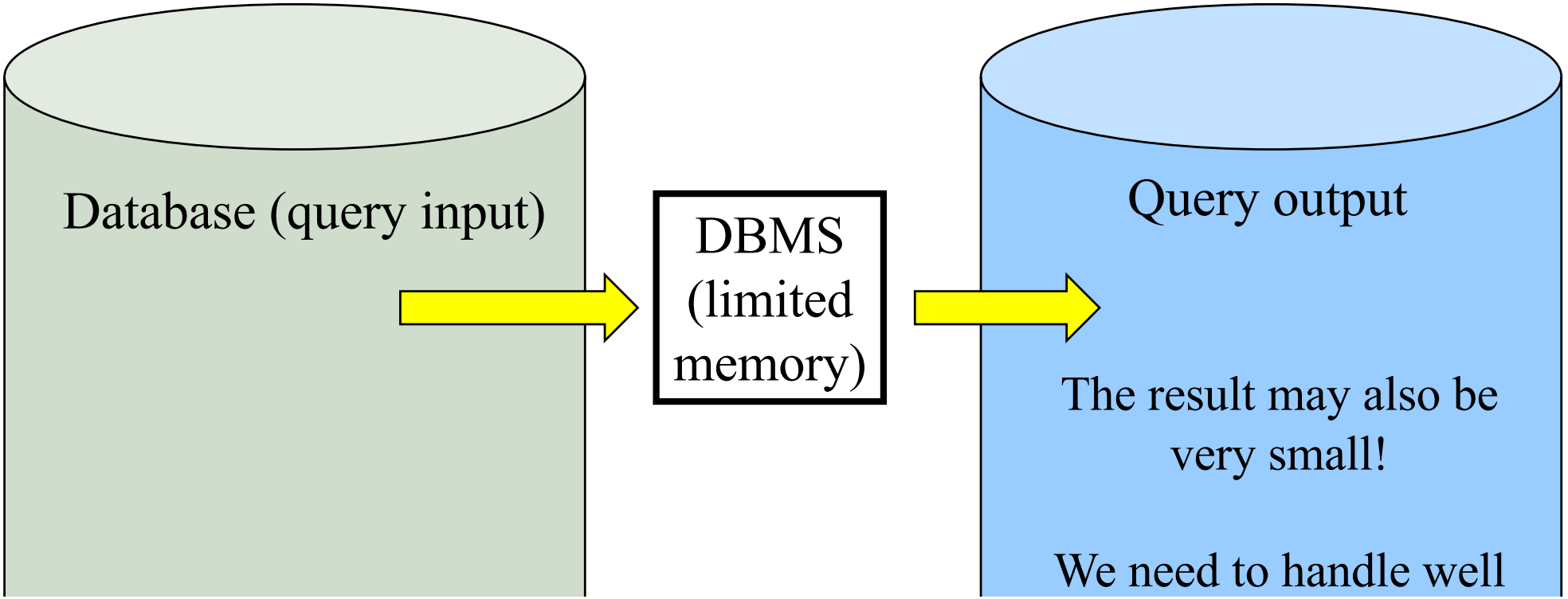
Memory

  - Each process may request data loads from disk to memory
- When the memory demand outgrows the available memory, the OS starts *swapping* processes to/from stable storage.
- If the Postgres server is swapped, query evaluation time may **explode** and be **totally unpredictable**



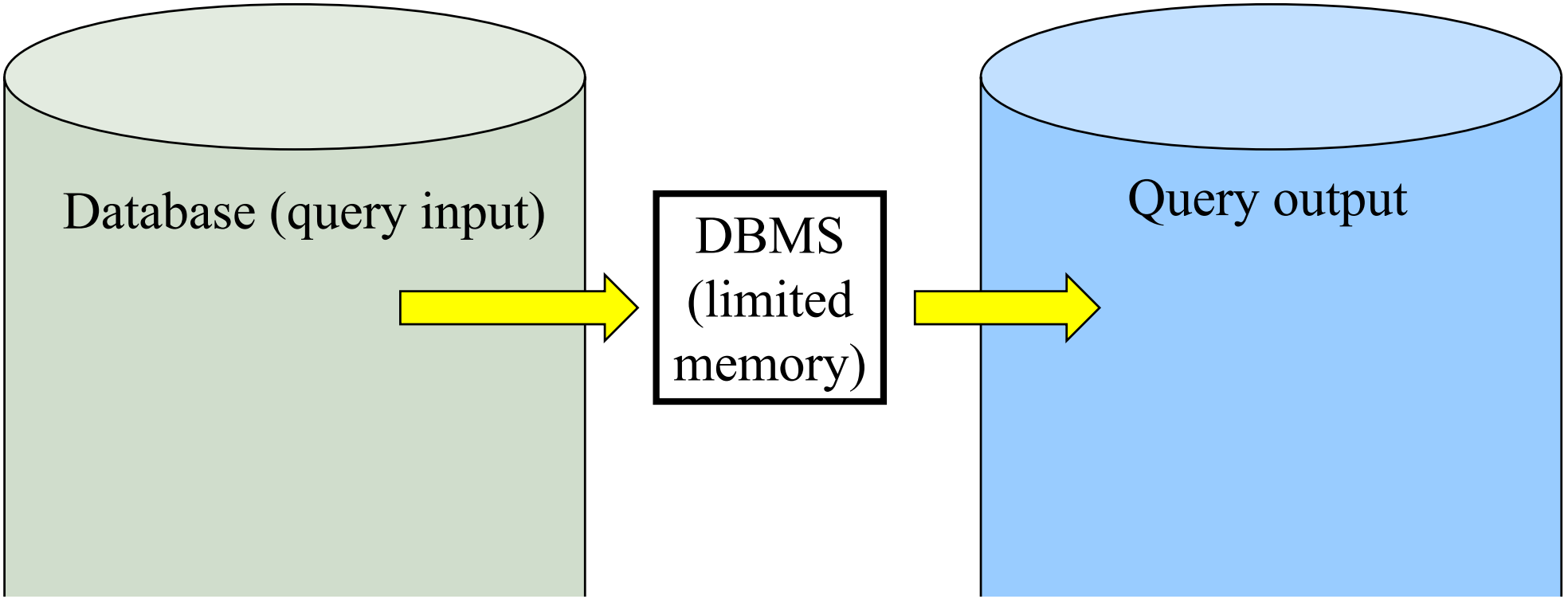
General goal: efficiently work on data that does not fit in memory

- Details of swapping policies may depend on the OS etc. but swapping is to be avoided
- To avoid causing OS swap, the server only runs what it can afford (knowing how much memory it has)
  - **All algorithms presented today aim to make this possible:**



General goal: efficiently work on data that does not fit in memory

- To avoid causing OS swap, the server only runs what it can afford (knowing how much memory it has)
- To avoid being a victim of OS swap (due to others), in production, a server should not share the machine with memory-hungry processes



Schema for Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)
Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: string)

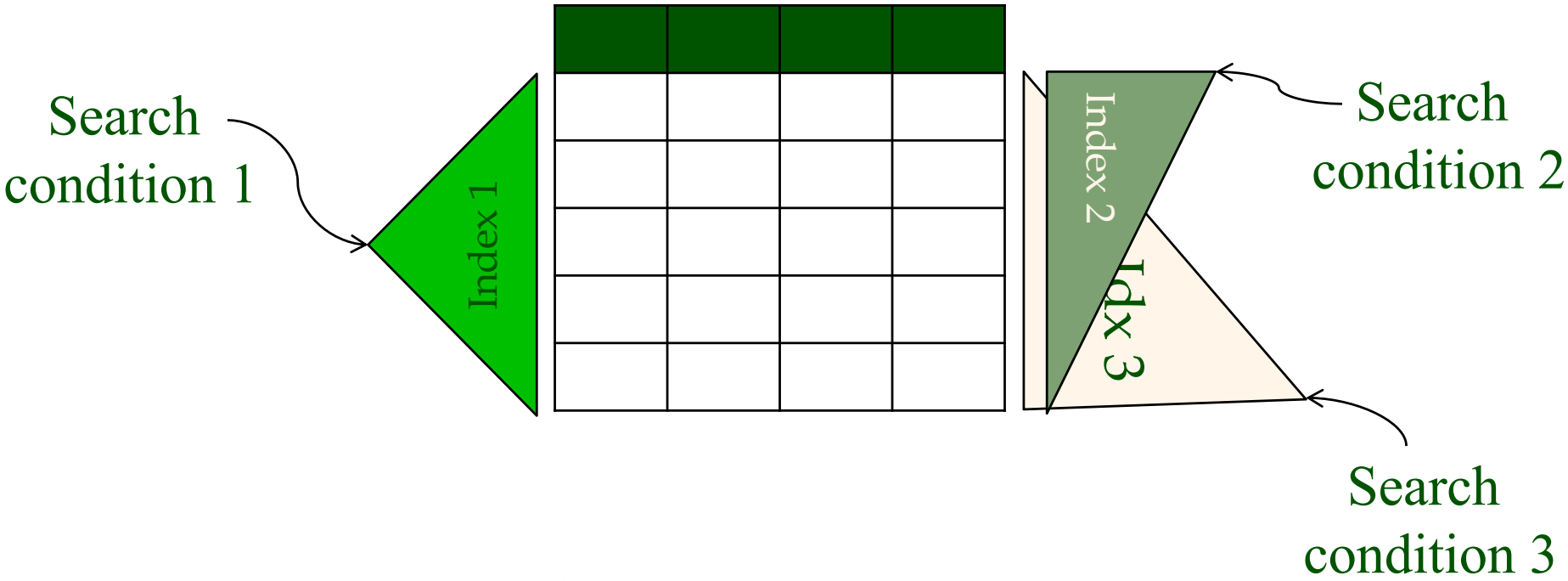
- Sailors S:
  - Each tuple is 50 bytes long,
  - 80 tuples per page,
  - 500 pages.
- Reserves R:
  - Each tuple is 40 bytes long,
  - 100 tuples per page,
  - 1000 pages.
- **Cost metric: # I/Os**

Outline

- Selections
- Sorting routine
- Joins
- Projections
- Set operators
- Group-by and aggregation

1 page in the memory buffer of DBMS is also called "buffer"; it has the size of 1 (disk) block

Implementing selections: basic idea



- *We always have*: the table
- *We may also have*: one or several indexes
- *We must chose*: the access method that minimizes cost, given the query conditions

Cost Model for Our Analysis

We ignore CPU costs, for simplicity:

- **B:** The number of data pages in a file
- **R:** Number of records per page
- **D:** (Average) time to read or write a disk page
- I/O cost: Measuring # of page I/O's
- Average-case analysis, based on simplistic assumptions.

Good enough to show the overall trends

Comparing File Organizations

- Heap files (random order)
- Sorted files, sorted on <*age, sal*>
- Clustered B+ tree file, Alternative 1 (the records are stored in the index leaves), search key <*age, sal*>
- Unclustered B + tree index on a heap file, search key <*age, sal*>

Assumptions in Our Analysis

- Heap Files:
  - New records inserted at the end of file.
- Sorted Files:
  - Files compacted after deletions.
- Indexes:
  - Alt (1): size of data entry = size of record
  - Alt (2), (3): size of data entry = 10% size of data record
  - B+Tree (Alt1):
    - 67% occupancy (typical): implies file size = 1.5 data size
    - Balanced with fanout F (133 typical) at each non-leaf level

Operations to Compare

- Scan: Fetch all records from disk
- Equality search on key: at most one match

  e.g., sid = 123
- Range selection:

  e.g., (age, sal) ∈ [(22, 20K), (30, 200K)]
- Insert a record

  e.g., (34, 'Bob', 7, 35)
- Delete a record

  e.g., (34, 'Bob', 7, 35)

Data access costs with a heap file

- Scan: B (# data pages in a file) * D (time to read a page)
- Equality search: on average 0.5 * B * D
- Range search: B * D
- Insert: read and write the last page → 2D
- Delete: search + D

B: # of pages / file | D: avg. time to R/W page | R: # records / page

Data access costs with a sorted file

- Scan: B*D
- Equality search: $\log_2(B) * D$
- Range search: $\log_2(B) * D$ + (D * # pages with matching records)
- Insert: search cost + B * D
- Delete: search cost + B * D

B: # of pages / file | D: avg. time to R/W page | R: # records / page

Data access costs with an unclustered B+ tree

Assumptions on the B+ tree index

- Pages are 67% full and a leaf page can hold pointers to 10 data pages →
  the number of leaf pages is (B / 10) / 67% ~ 0.15 * B
- Fan-out F=133 → height of the tree ~ $\log_F$ (0.15 * B)

- Scan: 0.15 * B * D + R * D (R number of records)
- Equality search: $\log_F$ (0.15 * B) * D + # pages with matching records * D
- Range search: $\log_F$ (0.15 * B) * D + # pages with matching records * D
- Insert: $\log_F$ (0.15 * B) * D + 2D (based on 67% assumpt.)
- Delete: $\log_F$ (0.15 * B) * D + 2D (based on 67% assumpt.)

B: # of pages / file | D: avg. time to R/W page | R: # records / page

Data access costs with a clustered B+ tree

Assumptions on the B+ tree index

- Pages are 67% full → There are 1.5*B leaf pages (storing the data)
- Fan-out F=133 → height of the tree ~ $\log_F (1.5 * B)$

- Scan: 1.5 * B * D
- Equality search: $\log_F (1.5 * B) * D$ + # pages with matching records * D
- Range search: $\log_F (1.5 * B) * D$ + # pages with matching records * D
- Insert: $\log_F (1.5 * B) * D + D$ (67% assumpt., only the index page to update)
- Delete: $\log_F (1.5 * B) * D + D$ (idem)

B: # of pages / file | D: avg. time to R/W page | R: # records / page

B: # of pages / file | D: avg. time to R/W page | R: # records / page

Data access costs: rough comparison

	Scan	**Equality search**	**Range search**	**Insert**	**Delete**
Heap file	B*D	0.5*B*D	B*D	2D	search + D
Sorted file	B*D	D*$\log_2$(B) + # pg. with matching records)	D*($\log_2$(B) + # pg. with matching records)	D*($\log_2$(B) + # pg. with matching records) + BD	D*($\log_2$(B) + # pg. with matching records) + BD
Unclustered B+ tree index	D*(0.15*B + R)	D*($\log_F$(0.15*B) + # pg. matching records)	D*($\log_F$(0.15*B) + # pg. matching records)	D*($\log_F$(0.15*B + 2)	D*($\log_F$(0.15*B) + 2)
Clustered B+ tree index	1.5*B*D	D*($\log_F$(1.5*B) + # pg. matching records)	D*($\log_F$(1.5*B) + # pg. matching records)	D*($\log_F$(1.5*B) + 1)	D*($\log_F$(1.5*B) + 1)

Data access costs / D (cost = # I/Os)

	Scan	**Equality search**	**Range search**	**Insert**	**Delete**
Heap file	B	0.5*B	B	2	search + 1
Sorted file	B	$\log_2$(B) + # pg. with matching records	$\log_2$(B) + # pg. with matching records	$\log_2$(B) + # pg. with matching records + B	$\log_2$(B) + # pg. with matching records + B
Unclustered B+ tree index	0.15*B + R	$\log_F$(0.15*B) + # pg. matching records	$\log_F$(0.15*B) + # pg. matching records	$\log_F$(0.15*B) + 2	$\log_F$(0.15*B) + 2
Clustered B+ tree index	1.5*B	$\log_F$(1.5*B) + # pg. matching records	$\log_F$(1.5*B) + # pg. matching records	$\log_F$(1.5*B) + 1	$\log_F$(1.5*B) + 1

Schema for Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)
Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: string)

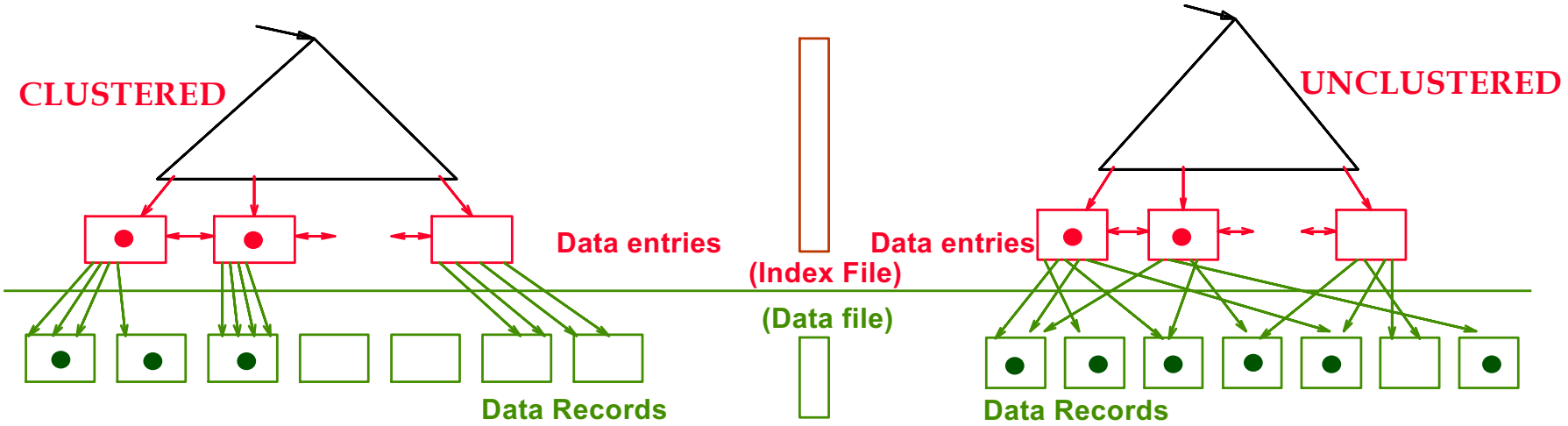
- Sailors S:
  - Each tuple is 50 bytes long,
  - 80 tuples per page,
  - 500 pages.
- Reserves R:
  - Each tuple is 40 bytes long,
  - 100 tuples per page,
  - 1000 pages.

Cost Factors of Selection

```
SELECT  *
FROM    Sailors S
WHERE   S.rating > 8
```

- Cost of selection includes:
  1) top down search in the index (usually approx., e.g., by 2)
  2) scan the relevant leaf nodes
  3) retrieve records from file
- Cost factor: *number of matching records*
  - *rating* > 8: if 20% of tuples match, 20%*500=100 pages, 80*100=8,000 tuples.
  - **Scanning leaf nodes**: if a data entry is 1/5 of a tuple, need 20 leaf nodes, so 20 I/Os.

Cost Factors of Selection (contd.)



- Cost factor: *clustering*
  - *rating* > *8*: 20% of tuples qualify, 100 pages, 8,000 tuples.
  - **Retrieving records from file ≈**
    - Clustered index: 100 I/Os.
    - Unclustered index: worst case 1 I/O per tuple; 8,000 I/Os!
    - Unclustered index + Sorting based on rid: ≤ 500 I/Os.
      (Bitmap Index Scan + Bitmap Heap Scan in Postgres)

Demo!

General Selections

- Boolean combination of predicates using AND and OR.
  - Conjunctive Normal Form (CNF), e.g.,
    - *pred1 AND (pred3 OR pred4)*
    - *(pred1 OR pred2) AND (pred3 OR pred4)*
- *File scan* always works for general selections.
- *Index scan* works when it matches a predicate that is a conjunct of CNF.
  - E.g., an index matching *pred1* can be used for *pred1 AND (pred3 OR pred4)*

Conjunctive Predicates Only

- CNF without OR: e.g. *pred 1 AND pred 2 AND pred 3*
  - Find the ***most selective access path***, retrieve tuples using it
    - File scan or index scan that gives the smallest I/O cost.
  - Apply remaining terms that don't match index *on the fly*.
    - Other terms do not affect I/O cost.

Example: SELECT * FROM Reserves
WHERE *day<8/9/94 AND bid=5 AND sid=3*

- Hash index on <*bid, sid*>: check *day<8/9/94* on the fly.
- B+ tree index on *day*: apply *bid=5* and *sid=3* on the fly.

Outline

- Selections ✔
- Sorting routine
- Joins
- Projections
- Set operators
- Group-by and aggregation

1 page in the memory buffer of DBMS is also called *buffer*; it has the size of 1 (disk) block

Number of buffers noted *B*

Why Sort?

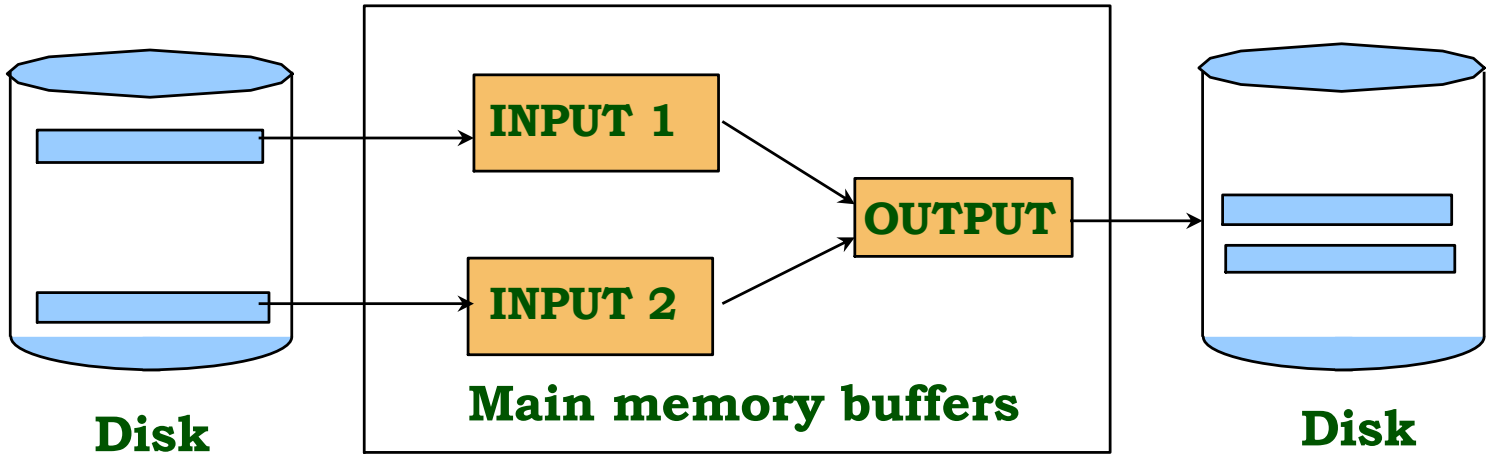
- Important utility in DBMS:
  - *Sort-merge* join algorithm involves sorting.
  - *Eliminate duplicates* in a collection of records (e.g., SELECT DISTINCT)
  - Request data in *sorted order* (e.g., ORDER BY)
    - e.g., find students in decreasing order of *gpa*
  - Sorting is first step in *bulk loading* B+ tree index.

- **Challenge**: the DBMS must be able to sort any volume of data, whereas memory size is always limited in practice.
  - For efficiency, it is crucial to **minimize # I/Os**

2-Way Sort: Requires 3 Buffers

- Pass 1: For each page, read the page, sort it, write it to disk.
  - Only one buffer page is used

- Pass 2, 3, …, etc.: Merge two sorted subfiles
  - Three buffer pages used.



Two-Way External Merge Sort

- *Divide and conquer*: sort subfiles (runs) and merge

A file of N pages:

Pass 1: N sorted runs of 1 page each

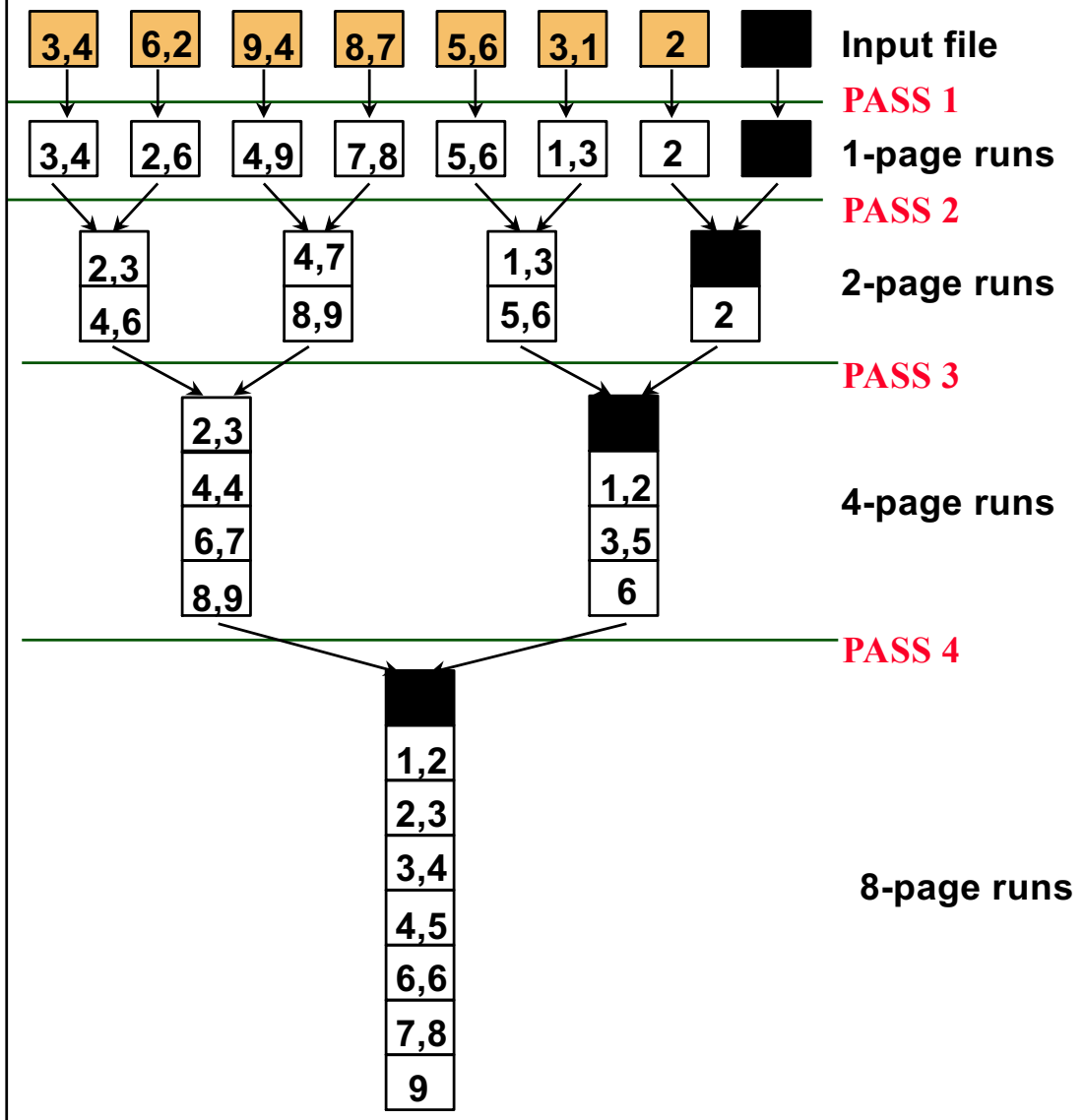
Pass 2: N/2 sorted runs of 2 pages each

Pass 3: N/4 sorted runs of 4 pages each

…

Pass P+1: 1 sorted run of 2^P pages

$P \geq \log_2 N$



Two-Way External Merge Sort

- *Divide and conquer*: sort subfiles (runs) and merge

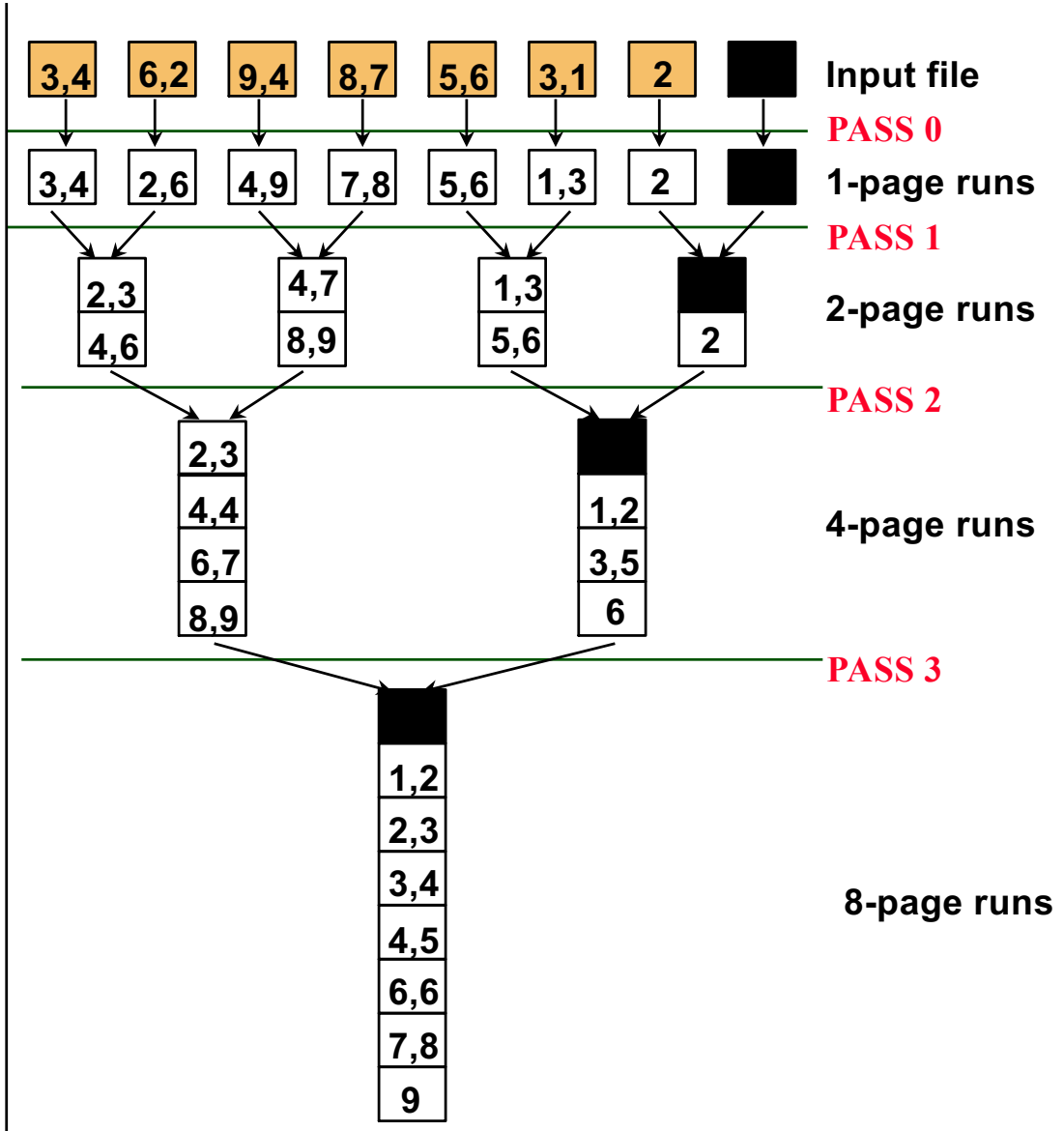
- Each pass, read + write N pages in file → 2N.

- Number of passes is:

$$\lceil \log_2 N \rceil + 1$$

- So total cost is:

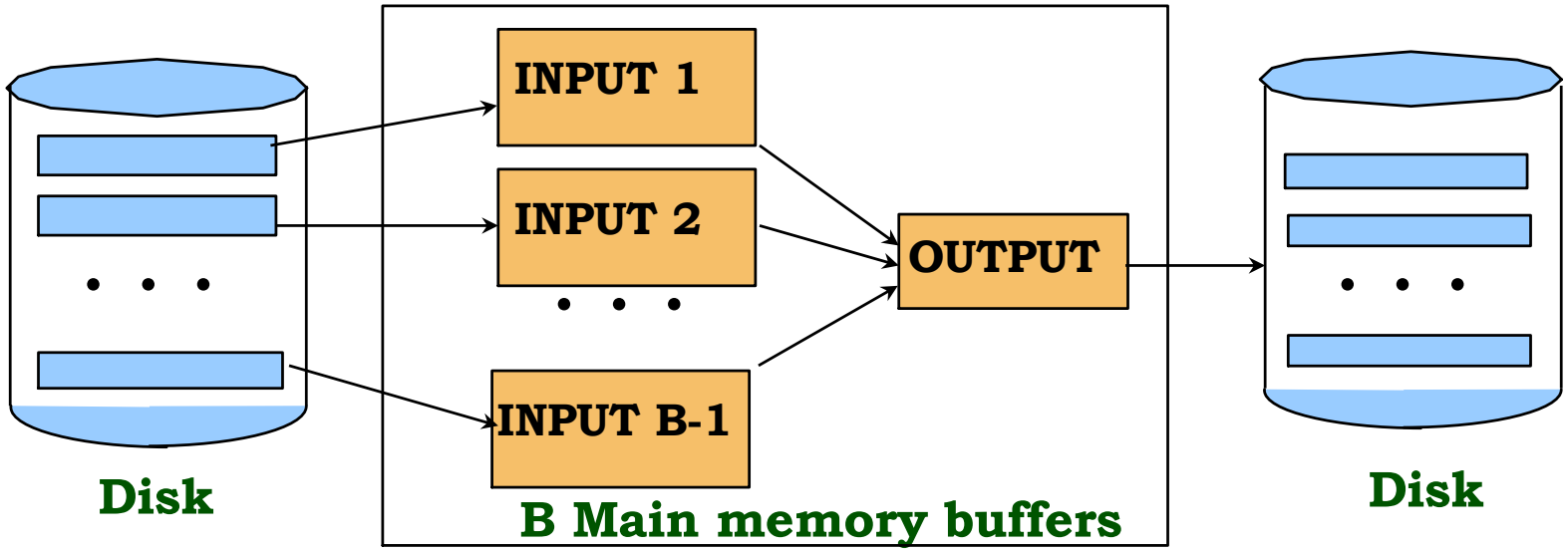
$$2N\left(\lceil \log_2 N \rceil + 1\right)$$



General External Merge Sort

Given B (>3) buffer pages, how to use them?

- **Pass 1**: Use *B* buffer pages. Produce ⌈N/B⌉ sorted runs of *B* pages each.
- **Pass 2, 3…, etc.**: Merge *B-1* runs
  - Read one page at a time from each run, merge in memory, gradually produce and write merged longer run



Cost of External Merge Sort

- E.g., with B=5 buffer pages, sort file of N=108 pages:

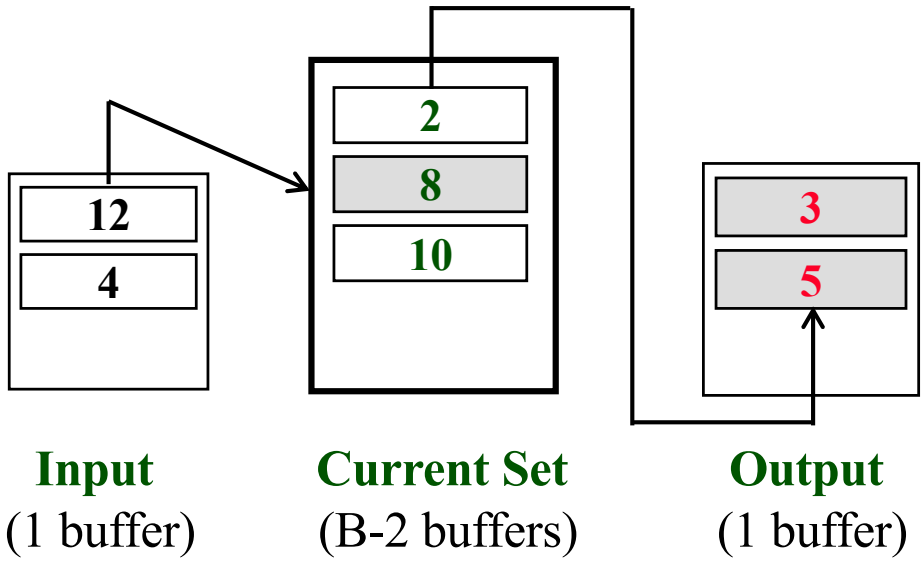
Pass 1	$\lceil 108/5 \rceil$ = 22 sorted runs of 5 pages each (last run is only 3 pages)	$\lceil N/B \rceil$ sorted runs of B pages each
Pass 2	$\lceil 22/4 \rceil$ = 6 sorted runs of 20 pages each (last run is only 8 pages)	$\lceil N/B \rceil / (B-1)$ sorted runs of B(B-1) pages each
Pass 3	2 sorted runs, 80 pages and 28 pages	$\lceil N/B \rceil / (B-1)^2$ sorted runs of $B(B-1)^2$ pages
…	…	…

- Number of passes = $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil$

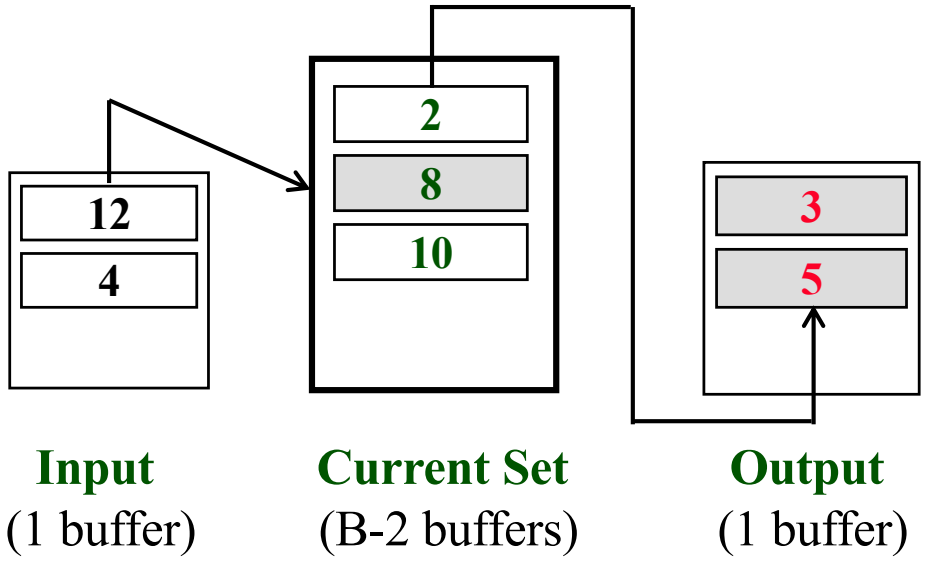
  Cost = $2N * (1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil)$

Replacement Sort: record-level algorithm optimization

- *Replacement Sort*: Produce initial sorted runs as long as possible. Used in Pass 1 of external sorting!
- Organize B available buffers:
  - 1 buffer for *input*
  - B-2 buffers for *current set*
  - 1 buffer for *output*

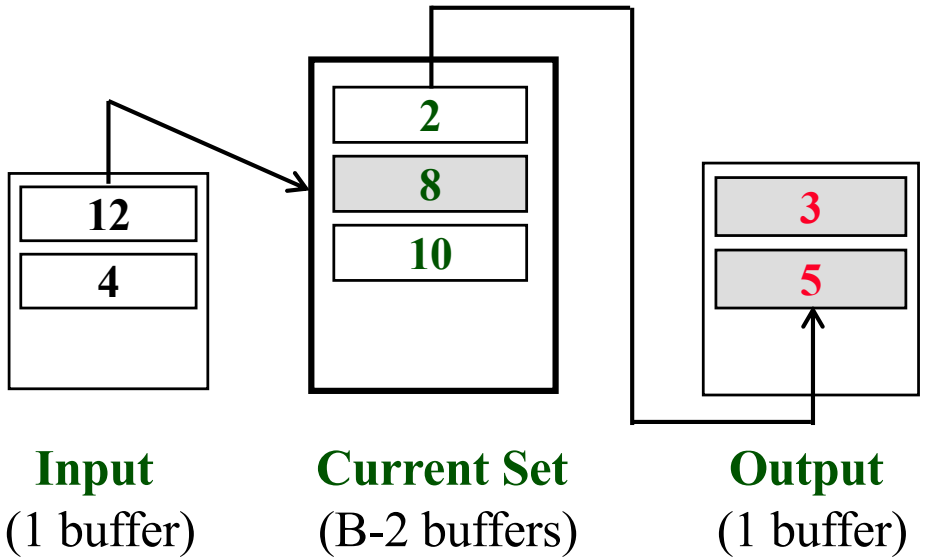


Replacement Sort: record-level algorithm optimization



- Pick tuple *r* in the current set (CS) s.t. *r* is the *smallest value* in CS that is ≥ *largest value in output*, e.g. 8, to extend the current run.
- Write output buffer out if full, extending the current run.
- Fill the space in current set by adding tuples from input.
- Current run terminates if *every tuple in the current set is < the largest tuple in output*.

Replacement Sort: record-level algorithm optimization



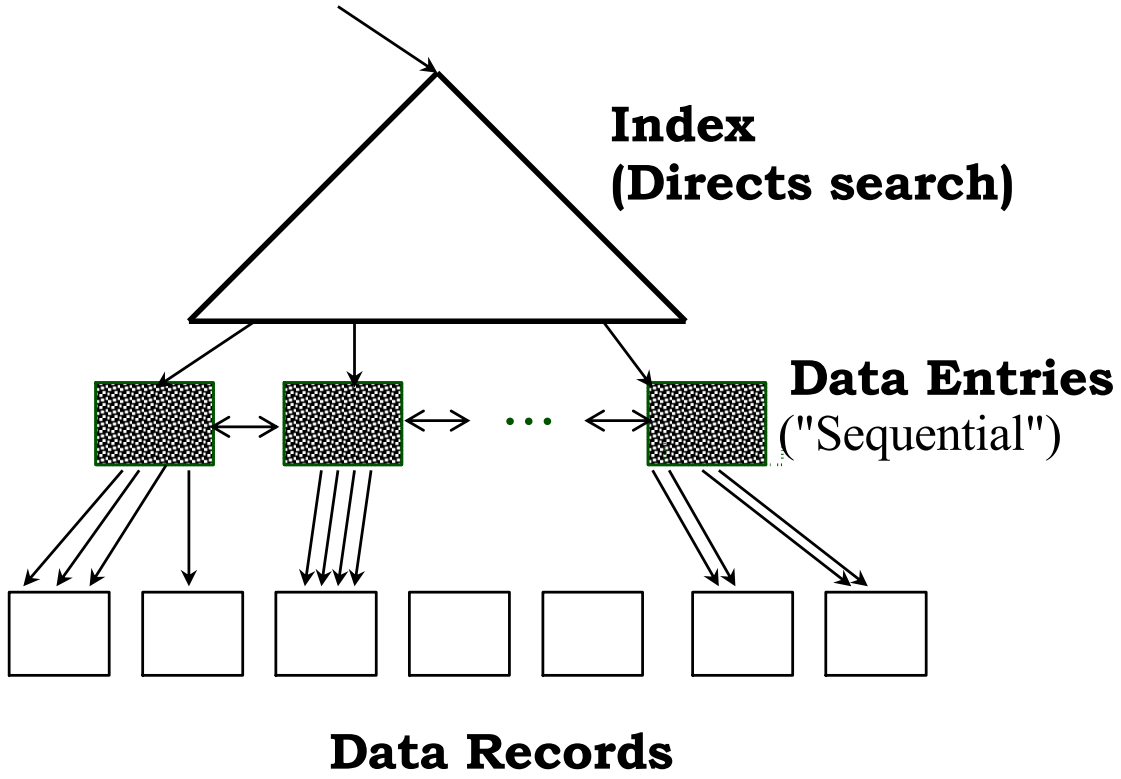
- When used in Pass 1 for sorting, write out sorted runs of size 2B on average.
- Affects calculation of the number of passes accordingly.

Using B+ Trees for Sorting

- Scenario: Table to be sorted has a B+ tree index on sorting attribute(s).
  - Retrieve students in increasing order of *age*.
- **Idea**: Can retrieve records in order by traversing leaf pages.
- Is this a good idea? Cases to consider:
  - B+ tree is clustered — ***Good idea***
  - B+ tree is not clustered — ***Could be a very bad idea***

Clustered B+ Tree Used for Sorting

- Alternative 1: cost of retrieving all index leaf pages
- Alternative 2: also cost of retrieving data records, but reading each page just once.



Almost always better than external sorting

Unclustered B+ Tree Used for Sorting

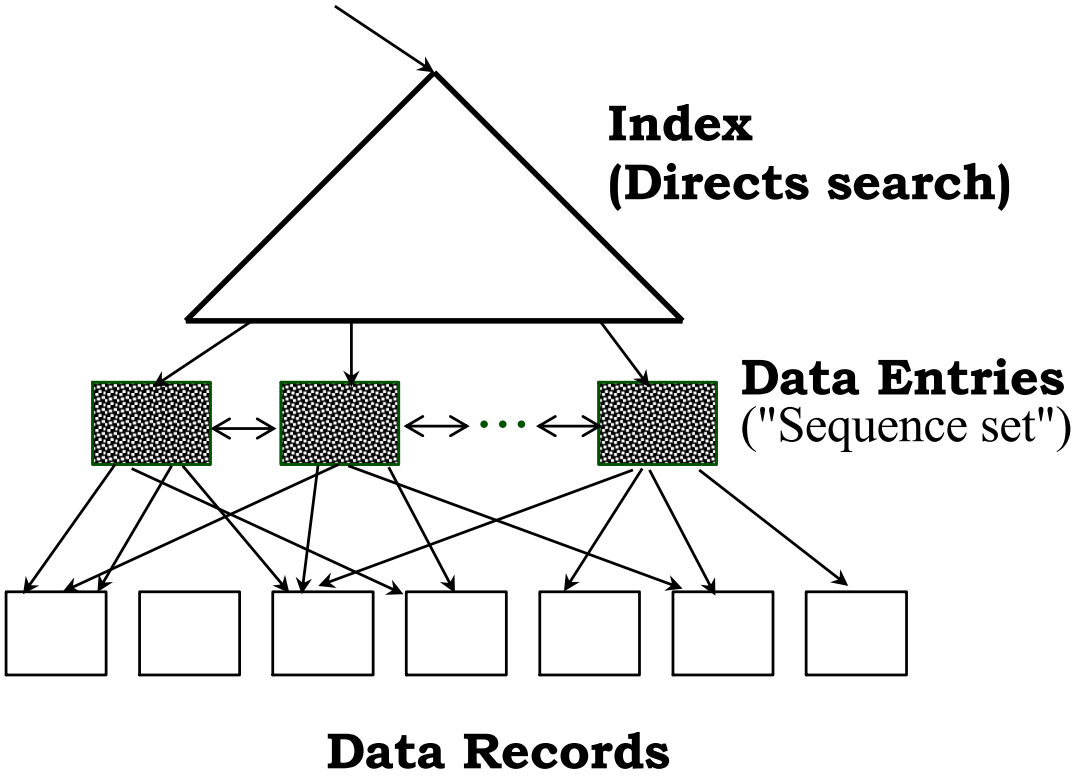
- Alternative 2: each data entry contains *rid* of a data record. In general, one I/O per data record



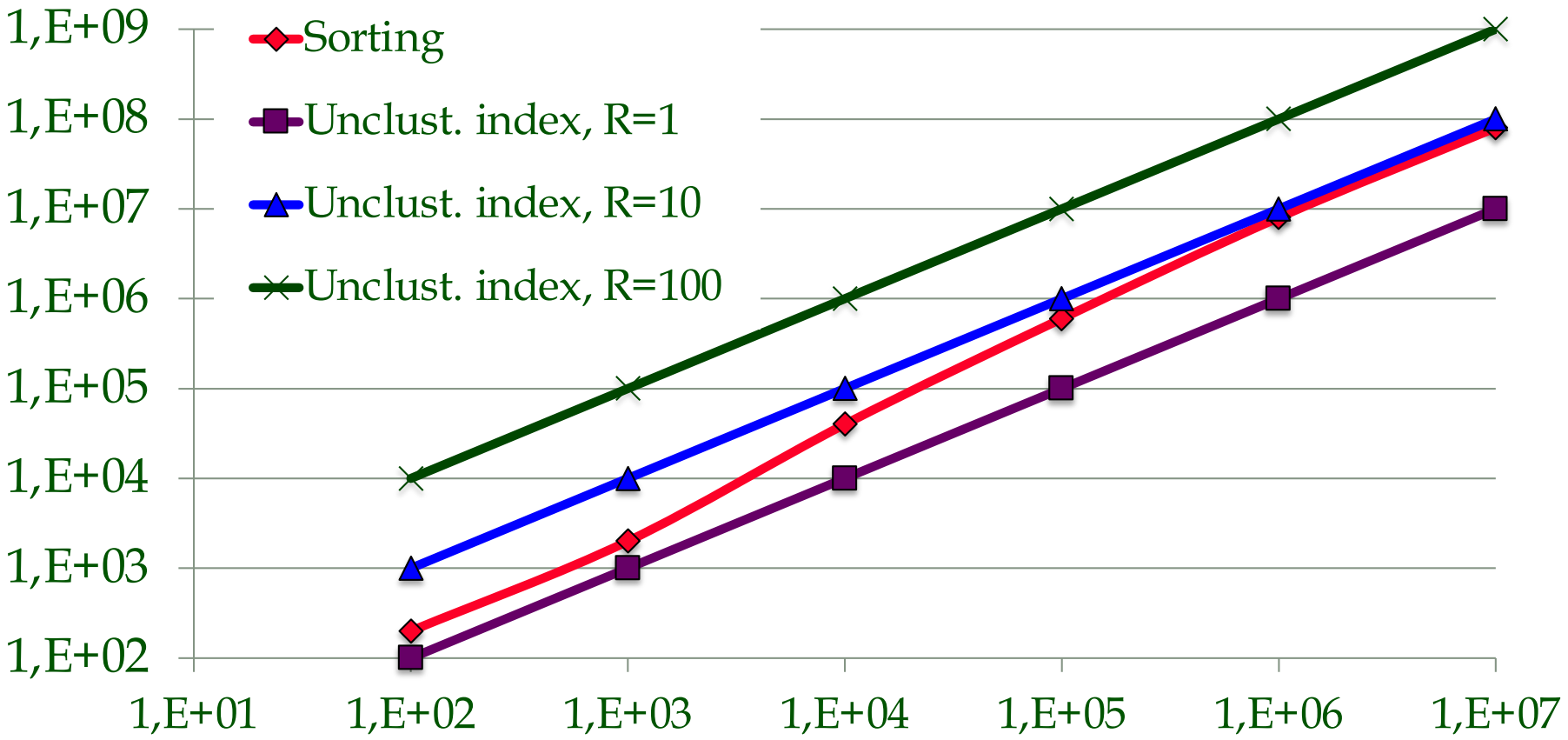
Worse case I/O: *RN*

R: # records per page

N: # pages in file



External Sorting vs. Unclustered Index



For sorting
- B=1,000
- Block size: 32

- R: # of records per page
  R=100 is the more realistic value.
- Worst case numbers (RN) shown

Outline

- Selections ✔
- Sorting routine ✔
- Joins
- Projections
- Set operators
- Group-by and aggregation

Join algorithms

- According to the join predicate used:
  - Equi-joins (on one or several attributes)
    "R.a=S.b and R.c=S.d"
  - Other predicates, e.g., "R.a < S.b"
  - *Not all predicates from the query are used in a join!*
- According to the class of algorithm used:
  - **Nested loops**-based
  - **Hash**-based
  - **Index**-based
  - **Sort**-based
- Not all classes of algorithms work for all join predicates

Equality Joins With One Join Column

```
SELECT  *
FROM    Reserves R, Sailors S
WHERE   R.sid = S.sid
```

- R ⋈S, natural join. Very common operation
- Semantics: cross product (×) followed by selection (σ)
  - If R × S is large, R × S followed by a selection is inefficient.
  - Many alternative algorithms
- Ignore output cost in analysis (same for all algorithms)
- We assume:
  - Relation R: M=1000 pages, p_R=100 tuples per page
  - Relation S: N=500 pages, p_S=80 tuples per page.

Tuple-oriented Nested Loops Join

- The most naive implementation

```
foreach record r of R do
        foreach record s of S do
                write out each matching pair <r, s>
                //r is in R-page, s is in S-page
```

- Cost:
  - Scanning R: M I/Os
  - S is scanned M * p_R times, and each costs N → p_R * N * M
  - M + p_R * M * N = 1000 + 100*1000*500 = 50,001,000 I/Os.
  - For 1 I/O = 10 ms, this means 138 hours.

Page-Oriented Nested Loops Join

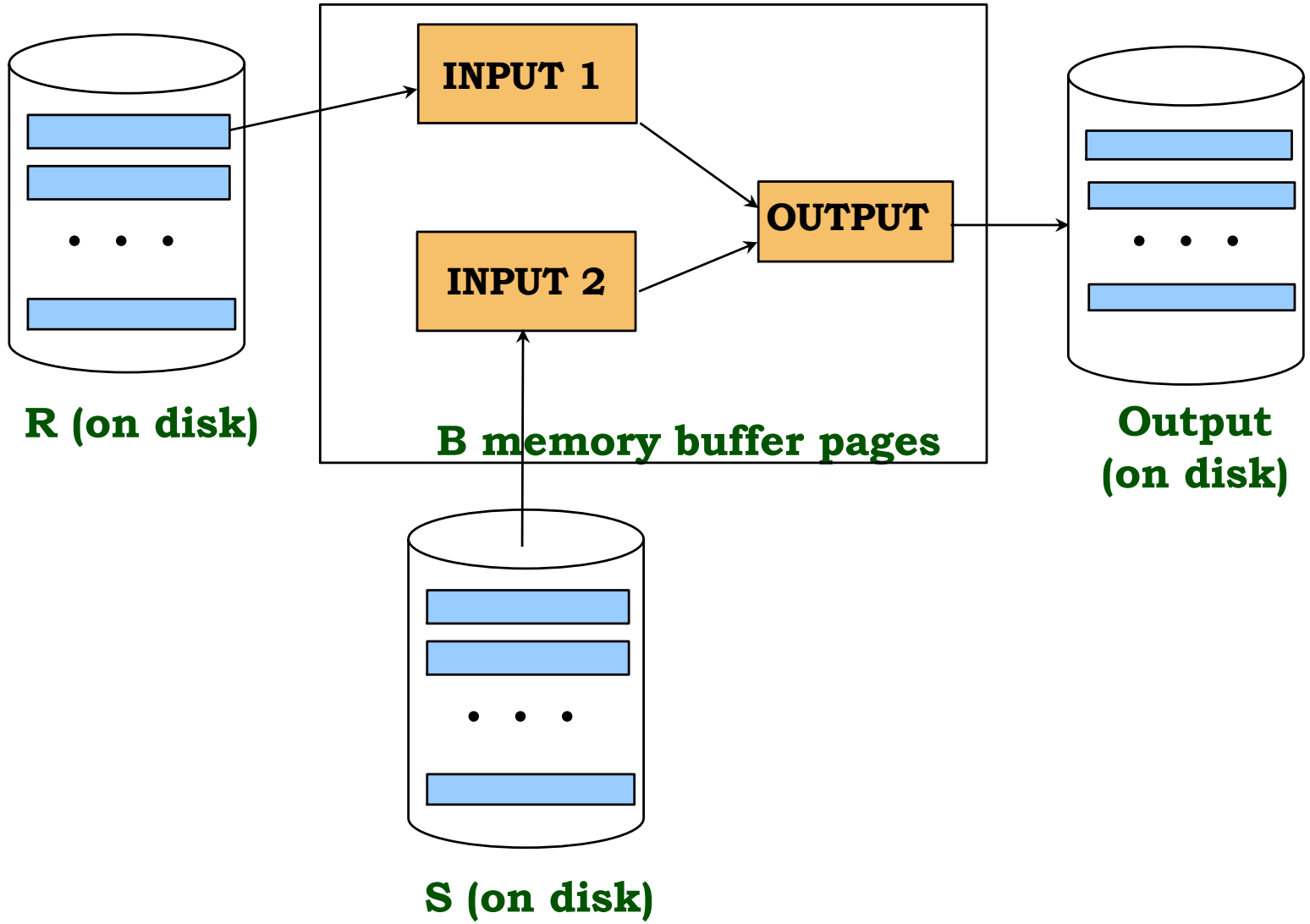
- A baseline approach:

```
foreach page of R do
        foreach page of S do
                write out each matching pair <r, s>
                //r is in R-page, s is in S-page
```

- Cost: M + M * N = 1000 + 1000*500 = 501,000 I/Os
  - For 1 I/O = 10 ms, this is 1.38 hours
  - 2M random I/Os; others are sequential I/Os.

- How many buffer pages do we need?

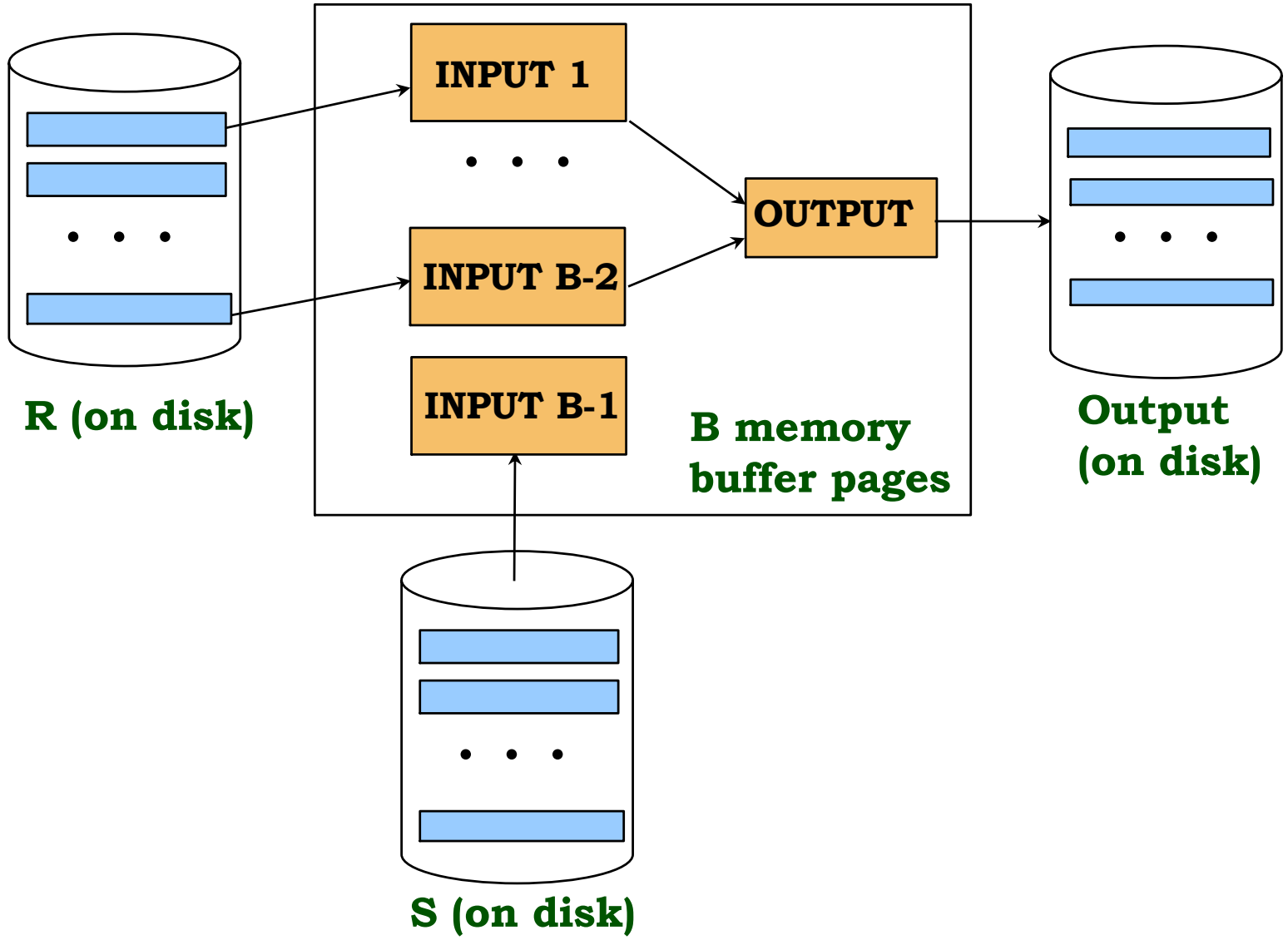
Page-Oriented Nested Loops Join



Block Nested Loops Join

- How can we utilize additional buffer pages?
  - If the smaller reln, say R, fits in memory, use R as outer, **read the inner S only once.**
  - Otherwise, read a big chunk of R each time, hence reducing # times S is read.
- Block Nested Loops Join:
  - The smaller relation R as outer, the other S as inner.
  - Buffer allocation:
    - 1 buffer page for scanning the inner S
    - 1 buffer page for output
    - *All remaining buffer pages for holding a "block" (chunk) of outer R*
      - "block" (chunk) size defined by: all the buffer space we can get!
      - Not to be confused with a block on disk

Block Nested Loops Join



Cost of Block Nested Loops Join

- Cost: Scan of outer + #outer blocks * scan of inner
  - B buffer pages available
  - Cost = size of outer + ⌈size of outer / B-2⌉ * size of inner

- E.g. B=102, Sailors S = 500 pages, Reserves R = 1000 pages.
  - What is the cost if S is outer, R is inner?
    - A block = B-2 = 100 pages
    - Cost = 500 + ⌈500/100⌉ * 1000 = 5,500 I/Os = 55 seconds
  - What is the cost if R is outer, S is inner?
    - Cost = 1000 + ⌈1000/100⌉ * 500 = 6,000 I/Os = 1 minute
- If each relation fits in memory, the smaller should be outer
- If smaller relation fits but not the larger, smaller → inner

Index Nested Loops Join

- Given an index on the join column of one relation, say S:

```
foreach tuple r in R do
        foreach tuple s in S where r == s (via index lookup) do
                add <r, s> to result
```

- Cost: M + (M * p_R * cost of finding matching S tuples)
  1) Cost of search in S index:
     - *B+ tree*: 2-4 I/O to search + extra pages for matches.
     - *Hash index* (not seen in course): 1.2 I/O to search + extra
  2) Cost of retrieving matching S tuples (assuming Alt. 2 or 3):
     - *Clustered index*: one or a few I/O (typical).
     - *Unclustered*: up to 1 I/O per matching S tuple.

Examples of Index Nested Loops

Sailors ⋈ Reserves

- Sailors: tuple size is 50 bytes, 80 tuples per page, 500 pages.
- Reserves: tuple size is 40 bytes, 100 tuples per page, 1000 pages.

- Hash-index (Alt. 2) on *sid* of Sailors (*primary key index*):
  - Scan Reserves: 1000 page I/Os, 100*1000 tuples.
  - For each Reserves tuple: # of matching Sailors tuples = 1
    - 1.2 I/Os to get data entry in index
    - 1 I/O to get the *exactly one* matching Sailors tuple.
  - Total: 1000+ 100*1000*2.2 = 221,000 I/O's.

Examples of Index Nested Loops

Sailors ⋈ Reserves

- Sailors: tuple size is 50 bytes, 80 tuples per page, 500 pages.
- Reserves: tuple size is 40 bytes, 100 tuples per page, 1000 pages.

- Hash-index (Alt. 2) on *sid* of Reserves:
  - Scan Sailors: 500 page I/Os, 80*500 tuples.
  - For each Sailors tuple: # of matching Reserves tuples = ?
    - Uniform distribution: 2.5 Reserves tuples/sailor (100,000/40,000).
    - 1.2 I/Os to find the index page with data entries.
    - Cost of retrieving the tuples is 1 or 2.5 I/Os (*cluster* or not).
  - Total: 500+80*500*(2.2~3.7) = 88,500~148,500 I/Os.

Sort-Merge ($R \bowtie S$) *for Equi-Join*

- Sort R and S on join column using external sorting.
- Merge R and S on join column, output result tuples.

Repeat until either R or S is finished:

- *Scanning*:
  - Advance scan of R until current R-tuple >=current S tuple,
  - Advance scan of S until current S-tuple >=current R tuple;
  - Do this until current R tuple = current S tuple.
- *Matching*:
  - Match all R tuples and S tuples with same value (called R-group and S-group of the current value).
  - Output <r, s> for all pairs of such tuples.

May need to read repeatedly some S-groups, once per matching R tuple

Example of Sort-Merge Join

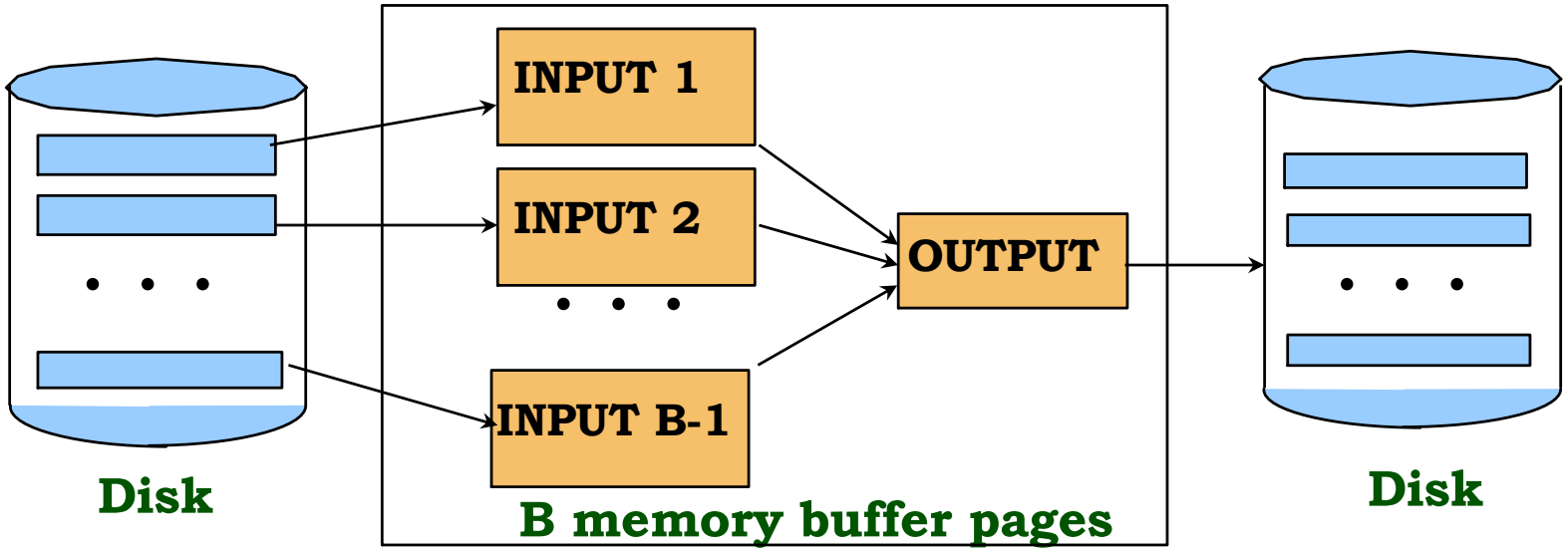
sid	sname	rating	age
22	dustin	7	45.0
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	bid	day	rname
28	103	12/4/96	guppy
28	103	11/3/96	yuppy
31	101	10/10/96	dustin
31	102	10/12/96	lubber
31	101	10/11/96	lubber
58	103	11/12/96	dustin

- Cost: Sorting_cost(R) + Sorting_cost(S) + Merging_cost
  - Merging cost ∈ [M+N, M*N]
  - M+N: *foreign key join* with S the referenced relation
    - No repeated reading of S
  - M*N: uncommon but possible.
- Needs at least 3 buffers; more can be used to avoid repeated reads of S tuple groups

Refinement of Sort-Merge Join

- Is there a guaranteed **linear-time** algorithm?

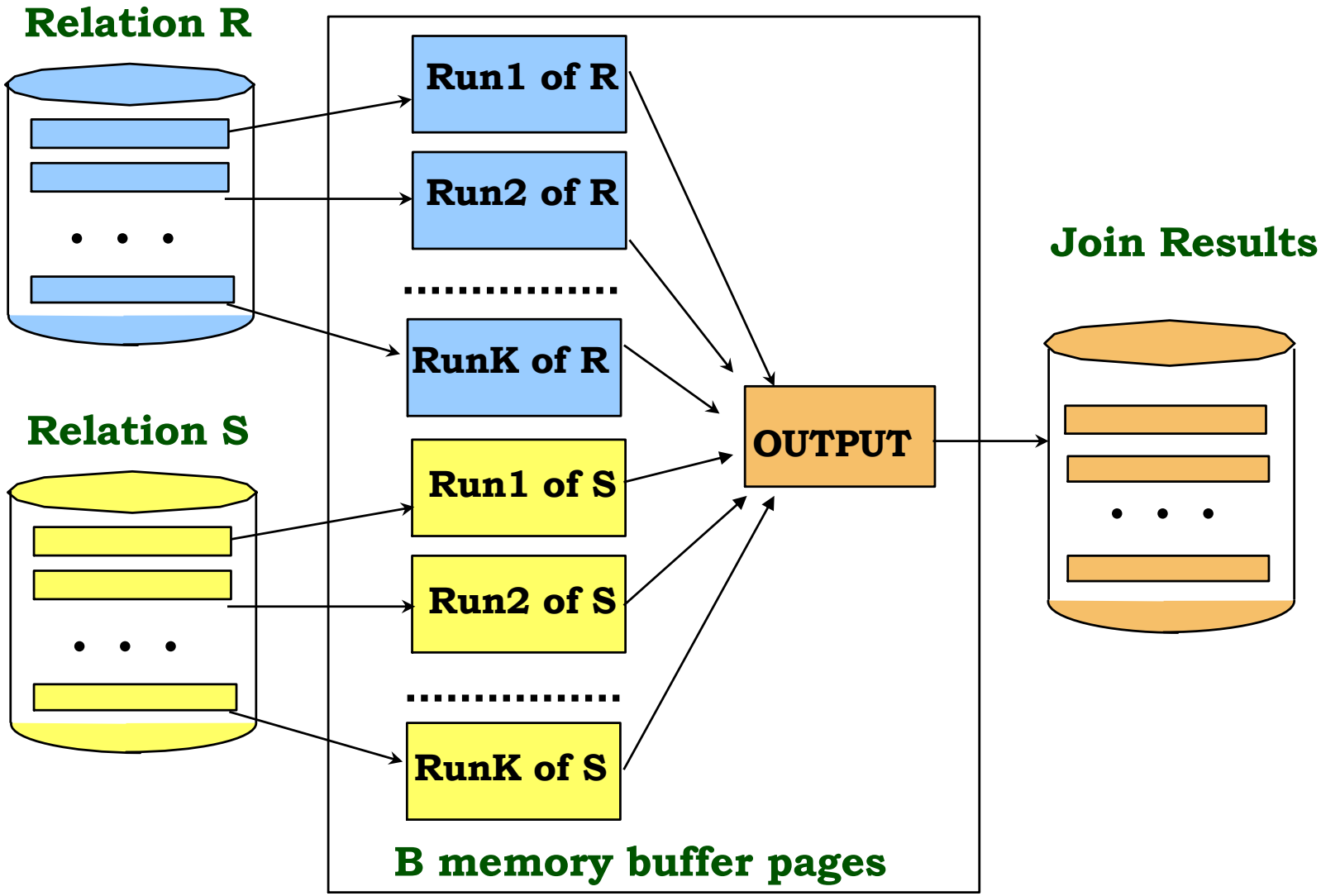


- Observed repeated merging phases:
  - *Sorting* of R and S has respective merging phases.
  - *Join* of R and S also has a merging phase.
  - Combine all these merging phases!

Two-Pass Sort-Merge Join

- Pass 1 *Sorting*: sort subfiles of R and S individually
- Pass 2 *Merging*: merge sorted runs of R and S
  - merge sorted runs of R,
  - merge sorted runs of S, and
  - compare R and S tuples using the *join condition*.
- Assume that we don't have too many duplicates…

Merging in Two-Pass Sort-Merge



Memory Requirement and Cost

- Memory requirement for two-pass sort-merge:
  - Let *U* be the size of the *larger* relation, *U* = max(M, N).
  - *Sorting* pass produces sorted runs of size 2B. So, number of runs per relation ≤ U/2B.
  - *Merging* pass holds sorted runs of both relations and an output buffer. So,
    2*(U/2B) + 1 <= B → **B > $\sqrt{U}$**

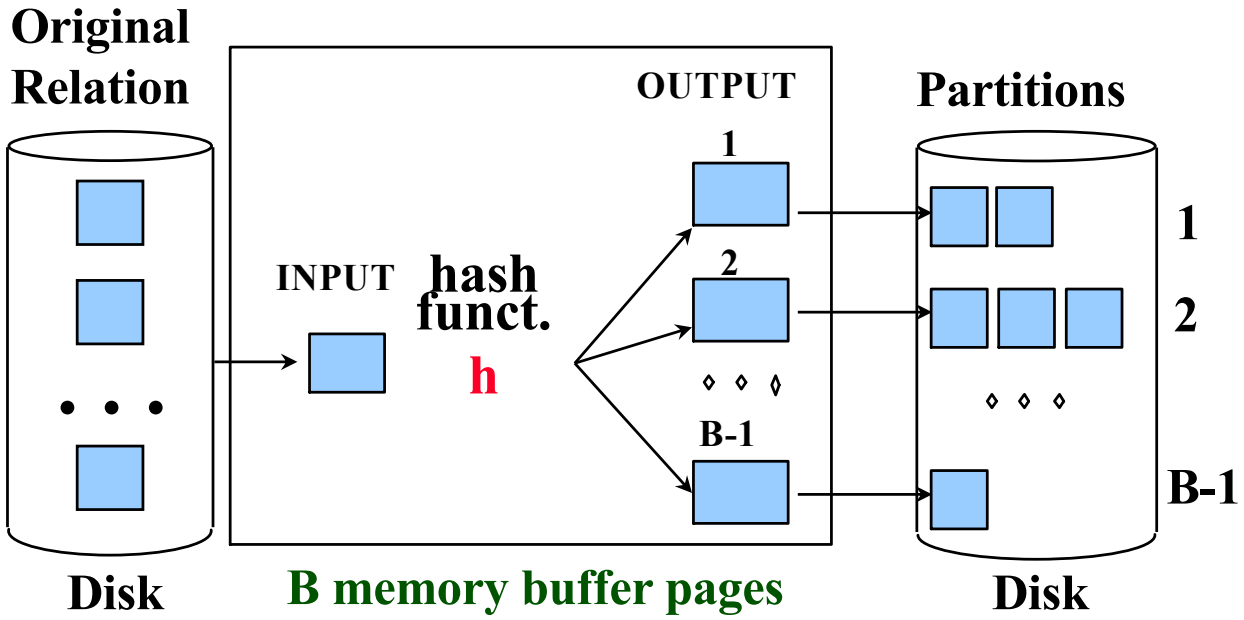
- Cost: read & write each relation in sorting pass
  + read each relation in merging pass
  (+ writing result tuples, ignore here) = 3 (M+N) !

Hash-Join for Equi-Join

- **Idea**: For an *equi-join*, partition both R and S using a hash function to make fewer tuple comparisons.
- Hash function:
  - given an input (e.g., an attribute value)
  - returns a value in an interval (usually [0... 2^k-1])
  - such that tuples are *close-to-uniformly distributed* in the hash function range
- The hash function can be seen as creating 2^k groups (or *buckets*)
  - Bucket i comprises all the tuples for which h(t.a)=i
  - Ideally, buckets have about the same size

Hash-Join for Equi-Join

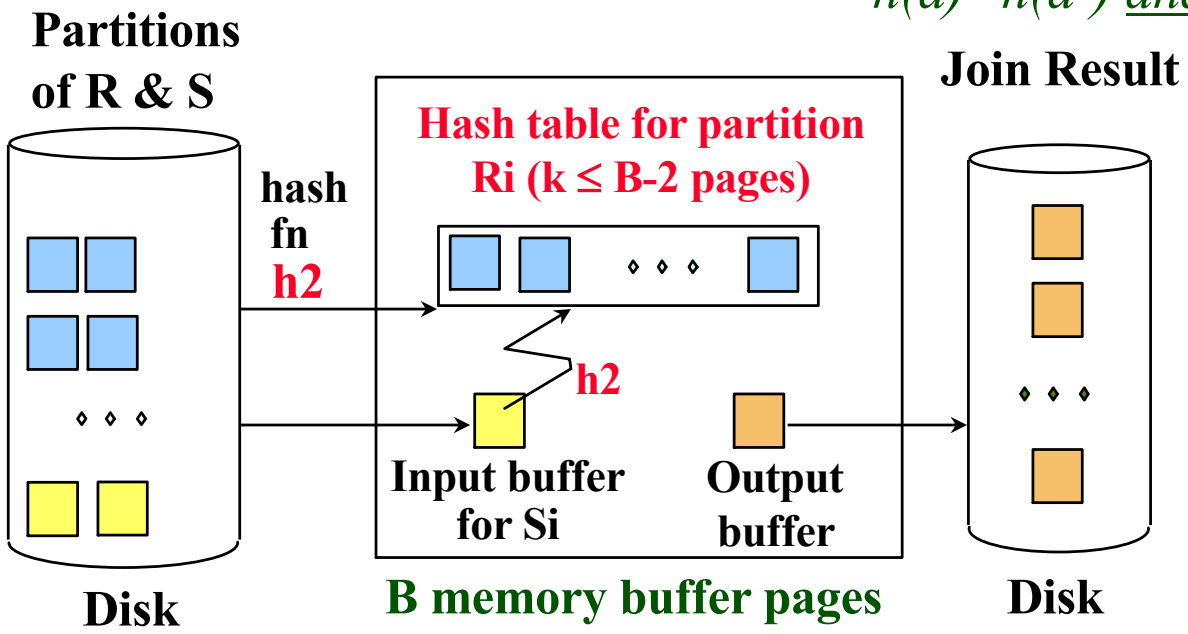
- **Idea**: For an *equi-join*, partition both R and S using a hash function to make fewer comparisons.
- Phase 1 *Partitioning*: Partition both relations using hash function **h** on join attributes (Ri tuples will only match with Si tuples).



Hash-Join

- Phase 2 *Probing*:
  - Read in partition Ri, build hash table using **h2 (<> h!)**
    - To distribute partition Ri within memory
  - Scan partition Si, one page at a time, apply h2, search for matches.

We only compare tuples such that h(a)=h(a') and h2(a)=h2(a')



Memory Requirement

- Partitioning: # partitions in memory ≤ B-1,

  Probing: to fit each Ri in memory, size of partition ≤ B-2.
  - A little more memory needed to build hash table, but ignored here.
- Assuming uniformly sized partitions, L = min(M, N):
  - L / (B-1) ≤ (B-2) → B > $\sqrt{L}$
  - Use the *smaller* relation as the building relation in probing phase.
- What if hash fn h does not partition uniformly?
  - One or more R partitions may not fit in memory.
  - Can apply hash-join recursively to this R-partition and the corresponding S-partition. Higher cost, of course…
- Sequential reads of both relations

Cost of Hash-Join

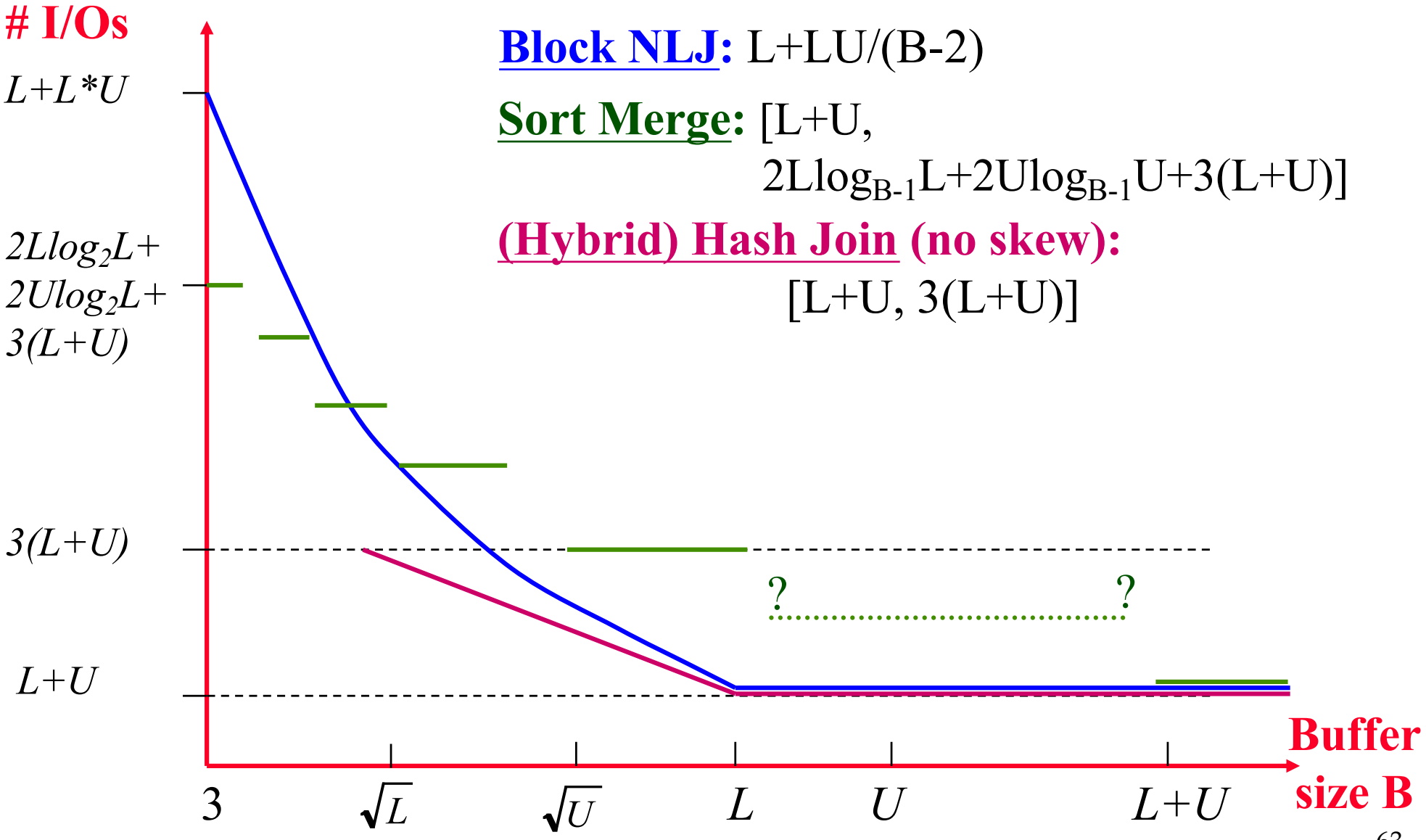
- Partitioning: reads+writes both relns; 2(M+N).

  Probing: reads both relns; M+N I/Os.

  Total cost = 3(M+N).
  - In our running example, a total of 4500 I/Os using hash join (compared to 501,000 I/Os w. Page NLJ).

- Sort-Merge Join vs. Hash Join:
  - Given a minimum amount of memory (*what is this, for each?*) both have a cost of 3(M+N) I/Os.
  - Hash Join superior on this count if relation sizes differ greatly. Assuming M<N, what if sqrt(M) < B < sqrt(N)?
  - Sort-Merge less sensitive to data skew; result is sorted.

Rough Comparisons of Join Methods



Block NLJ: L+LU/(B-2)

Sort Merge: [L+U, $2L\log_{B-1}L+2U\log_{B-1}U+3(L+U)$]

(Hybrid) Hash Join (no skew): [L+U, 3(L+U)]

General Join Conditions

- Equalities over several attributes (e.g., *R.sid=S.sid* AND *R.rname=S.sname*):
  - ✓ Block NL works fine.
  - ✓ For Index NL,
    - use index on <*sid, sname*> if available; or
    - use an index on *sid* or *sname*, check the other predicate on the fly.
  - ✓ For Sort-Merge and Hash Join, sort/partition on combination of the two join columns.

General Join Conditions

- Inequality conditions (e.g., *R.rname < S.sname*):
  - ✓ For Index NL, need B+ tree index.
    - Range probes on inner; number of matches likely to be much higher than for equality joins.
    - Clustered index is much preferred.
  - ✓ Block NL often works well.
  - ✗ Hash Join, Sort Merge Join not applicable.

Outline

- Selections ✔
- Sorting routine ✔
- Joins ✔
- Projections
- Set operators
- Group By aggregation

The Projection Operation

```
SELECT  [DISTINCT] R.sid, R.bid
FROM    Reserves R
```

- Projection consists of two steps:
  - Remove attributes not in the projection list (trivial).
  - If DISTINCT is specified, eliminate any duplicate tuples.
- Algorithms for projection with distinct:
  - single-relation sorting
  - hashing based on all remaining attributes.

Projection Based on Sorting

- *Sorting pass*: modified to remove unwanted fields.
  - Runs of about 2B pages are produced.
  - But tuples in runs are smaller than input tuples. (Size ratio depends on # and size of fields that are dropped.)

- *Merging passes*: modified to eliminate duplicates.
  - # result tuples smaller than input (depends on # of duplicates.)

Projection Based on Hashing

- *Partitioning phase*: Partition input relation using *h1*; for each tuple, discard unwanted fields.
  - Result is B-1 partitions (of tuples with only wanted fields).
  - Each partition can be loaded back in B-1 buffers.
  - 2 tuples from different partitions guaranteed to be distinct.

- *Duplicate elimination phase*: Read each partition, build an in-memory hash table using *h2* on *all* fields, discard duplicates.

Outline

- Selections ✔
- Sorting routine ✔
- Joins ✔
- Projections ✔
- Set operators
- Group-by and aggregation

Intersection, Union, Set Difference

- Intersection: special case of join, with equality on all fields

- Union and Set Difference input bags and output sets (duplicate elimination inside!)
- They can be implemented either through sorting or hashing (see next)

Union

- Union: Tuples in either reln. 1 or reln. 2.
  - Selects *distinct* values only.
  - Sorting based (over two relations):
    - Sort both relations on combination of *all* attributes.
    - Scan sorted relations, merge them and remove duplicates.
  - Hashing based (over two relations):
    - Partition R and S using *h1* on *all* attributes.
    - For each R-partition, (1) write it out (as part of the answer) and (2) build an in-memory hash table using *h2*. For each tuple in S-partition, probe the hash table.
      - If the S tuple is in the hash table, discard it.
      - Otherwise, output it.

Set Difference

- Set-difference: Tuples in reln. 1, but not in reln. 2.
  - Sorting-based solution, similar to Union.
  - Hashing-based solution, similar to Union.

Outline

- Selections ✔
- Sorting routine ✔
- Joins ✔
- Projections ✔
- Set operators ✔
- Group-by and aggregation

Aggregate Operations (AVG, MIN, etc.)

```
SELECT min(S.age)
FROM   Sailors S
WHERE S.rating = 10
```

- Aggregation without grouping
  - *File scan*: in general, requires scanning the relation.
  - *Index only scan*: if a tree index search key includes all attributes in the SELECT and WHERE clauses.
    - e.g. B+tree on <rating, age>

Aggregate Operations (contd.)

```
SELECT      min(S.age)
FROM        Sailors S
WHERE       S.rating > 5
GROUP BY S.rating
```

- Aggregation with grouping (GROUP BY)
  - *Single-relation sorting*: sort by group-by attribute(s); compute aggregate for each group in last merging phase.
  - *Single-relation hashing*: hash on group-by attribute(s); compute aggregate using in-memory hash table for each partition.
  - *Index only scan*: if a tree index search key includes all attributes in SELECT, WHERE and GROUP BY clauses.
    - e.g. B+tree on <rating, age>

Questions



Summary

- Algorithms for evaluating relational operators use some simple ideas extensively:
  - Indexing: Can use WHERE conditions to retrieve small set of tuples (selections, joins)
  - Iteration: Sometimes, faster to scan all tuples even if there is an index. (And sometimes, we can scan the data entries in an index instead of the table itself.)
  - Partitioning: By using sorting or hashing, we can partition the input tuples and replace an expensive operation by similar operations on smaller inputs.

Summary (Contd.)

- Many implementations for each operator; no universally superior technique for most operators.
- Must consider available alternatives for each operation in a query and choose best one based on:
  - available resources (fundamentally: memory) and
  - statistics (table size, # tuples matching value k).
- The choice is part of the broader task of optimizing a query composed of several operators.
  - To be seen